运输层

概述和运输层服务

- 运输层服务和协议
  - 提供两个端系统的应用进程之间的逻辑通信
  - 运输层协议工作在端系统
    - 发送方将报文封装为报文段，提交给网络层
    - 接收方将报文段解析成报文，提交给应用层
- 运输层和网络层的关系
  - 运输层
    - 向上面应用层提供通信服务
    - 属于面向通信部分的最高层
    - 同时是用户功能的最底层
    - **负责进程之间的逻辑通信**
  - 网络层
    - 负责主机之间的逻辑通信
- 运输层的服务
  - TCP
    - 握手机制（三次握手）：面向连接的逻辑通信
    - 流控制：发送速度不能大于接受速度
    - 拥塞控制：从发送端系统感知、避免网络拥塞
  - UDP
    - 不提供必要服务
    - 尽最大努力向网络交付数据
  - 运输层没有提供的服务
    - 最低时延服务
    - 最低宽带服务

多路复用和多路分解

- 端口
  - 使用16bit的整数来标识进程，每个端口都对应着一个进程
  - 在不同的主机上，相同的端口可能有不同的用途
- 套接字
  - 主机地址+端口号
- 多路复用
  - 从源主机的不同套接字中收集数据块，并为每个数据块封装上首部信息从而形成报文段，然后将其发送到网络层
  - 多路分解机制
    - 主机收到IP数据报
      - 数据报中有源IP和目的IP
      - 数据报中封装了应用报文
      - 每个报文段中都标识了源端口号和目的端口号
    - 定向传输给应用进程的套接字
- 多路分解
  - 将运输层报文段中的数据放置到正确的套接字上（数据流的分解）
- 无连接的复用和分解
  - 使用端口号创建套接字
  - UDP套接字使用二元组来标识-(dest IP address, dest port number)
  - 不同主机发送的报文，只要目的IP和目的端口号相同，都会提交给同一个套接字
- 面向连接的复用和分解
  - TCP的套接字使用四元组标识-(source IP address ,source port number,dest IP address,dest port number)
  - 服务器端的应用进程可能有并发的TCP套接字（即一个进程对应多个套接字）

UDP

- 概述
  - 基于UDP的应用
    - DNS
    - SNMP：简单网络管理协议
    - 流媒体传输
    - 因特尔电话
  - 基于UDP，如何实现可靠传输
    - 将可靠传输机制放在应用层
    - 在应用程序中定义差错控制的机制
- UDP报文段结构
  - 提供差错检验：检验和
    - 1、按每十六位求和得出一个三十二位的整数
    - 2、若这三十二位数的高十六位不为零，则高十六位加低十六位再得到一个三十二位的数
    - 3、重复第二步知道高十六位为零
    - 4、最后将低十六位按位取反得到检验和
  - 不提供差错控制，出现差错后直接丢弃数据报（且在传输过程中，若路由器缓存满了，多余的应用分组也会被丢弃）

拥塞控制原理

- 网络拥塞
  - 过多的端点高速发送大量的数据导致路由器难以胜任数据的转发
  - 表现
    - 数据分组的丢失（路由器缓存的溢出）
    - 产生较长时延（路由器缓存排队）
- 拥塞的原因和代价
  - 两个发送方和一台无线缓存的路由器
    - 带宽为R，没有重传，流控制和拥塞控制
    - 拥塞原因
      - 路由器缓存产生队列
    - 拥塞代价
      - 加长网络时延
  - 两个发送方和一台有限缓存的路由器
    - 有分组重传控制
    - 拥塞原因
      - 路由器缓存溢出，丢失分组
      - 路由器缓存产生队列
    - 拥塞代价
      - 发送方重传丢失分组
      - 发送方重传超时分组
  - 四台发送方和一台有限缓存路由器和多条路径
    - 新的拥塞代价
      - 一个路由器将分组丢失，则这个路径上先前的路由器都做了无用功
- 拥塞控制的方法
  - 问题的关键
    - 如何让发送方知道网络产生了拥塞
  - 方法
    - 端到端的拥塞控制
      - 网络层不提供显示拥塞反馈
      - 端点通过超时和冗余确认来感知网络是否发生了拥堵
      - 工作在运输层，通过TCP实现拥塞控制
    - 网络辅助的拥塞控制
      - 路由器会使用特殊的分组告知端点产生了拥塞
      - 工作在链路层

TCP拥塞控制

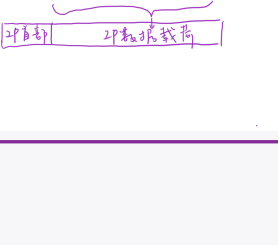
- 发送方感知网络拥塞并采取措施
  - 如何感知?
    - 发送方产生超时事件
    - 发送方收到三个冗余确认
  - 如何控制?
    - 发送方设置拥塞窗口
- ★ 拥塞窗口
  - 发送端的一个动态参数
  - 用于控制发送速度
  - 初始值为1MSS
- 拥塞窗口的动态调节策略
  - 在拥塞发生前-加性增
  - 在感知拥塞后-乘性减
- TCP拥塞控制算法
  - 缓慢启动
    - 初始速度很低，然后指数增长
      - 拥塞窗口初始值为1MSS
      - 每经历一个RTT，拥塞窗口就翻倍
      - 收到ACK后立即更新
    - 产生超时事件
      - 拥塞窗口重设为1MSS
      - 然后开始慢启动
    - 收到三个冗余确定
      - 拥塞窗口减半
      - 然后开始线性增长
  - 拥塞避免
    - 对于慢启动在丢包之前提前采取的措施
      - 设定一个阈值
      - 拥塞窗口达到阈值后进入加性增长
  - 快速恢复
    - TCP Tahoe（快速重传）
      - 产生超时事件
        - 阈值收缩为拥塞窗口的一半
        - cwnd=1，进入慢启动
      - 收到三个冗余确认
        - 阈值收缩为拥塞窗口的一半
        - cwnd=1，进入慢启动
    - TCP Reno（快速恢复）
      - 产生超时事件
        - 阈值收缩为拥塞窗口的一半
        - cwnd=1，进入慢启动
      - 收到三个冗余确认
        - 阈值收缩为拥塞窗口的一半
        - 策略1
          - cwnd=ssthresh+3，进入加性增长
          - cwnd=ssthresh，进入加性增长

可靠传输原理

- 可靠数据传输（rdt）概述
  - 不可靠的信道特点决定了可靠传输服务的复杂性
  - 方法：定义四个函数
- ★ 构造可靠的传输协议-有限状态机（FSM）（结合PPT Lecture009）
  - **停止等待协议/比特交替协议**
  - rdt 1.0
    - 前提
      - 下层信道是可靠的
      - 没有分组差错
      - 没有分组丢失
    - ★ 有限状态机
      - 发送端（一个状态）
        - 向底层发送数据，只需等待来自顶层的调用
      - 接收端（一个状态）
        - 从底层信道读取数据
    - 现实
      - 底层信道存在干扰，会带来bit差错
  - rdt 2.0
    - 着手处理bit差错
      - 如何检测bit差错：检验和
      - 如何恢复bit差错
        - 肯定确认：接收方告诉发送方数据分组正确到达
        - 否定确认：接收方告诉发送方数据分组出错，发送方收到后进行重传
      - 停止等待协议（ARQ）
        - 接收方检测
        - 接收方反馈
        - **发送端发送完后不能直接等待顶层调用，而是要等待接收端的确认信息**
    - ★ 有限状态机
      - 发送端（两个状态）
      - 接收端（一个状态）
    - 现实
      - 当接收方的确认信息出错怎么办?
      - 发送方是否重传?
      - 重传会不会导致接收方数据重复?
  - rdt 2.1
    - 处理出错的确认信息
      - 为数据分组编号
      - 接收方收到重复编号的数据后就起丢弃
      - 使用1个bit为数据分组编号，执行0和1之间的交替
    - ★ 有限状态机
      - 发送端（四个状态）
        - 必须检测确认信息是否出错
        - 状态数量必须翻倍：在状态值中必须记录当前数据分组是0还是1
      - 接收端（两个状态）
        - 必须检测数据分组是否出错
        - 必须检测数据分组是否重复：状态值暗示了期待的分组号
  - rdt 2.2
    - 不要否定确认（NAK）
      - 与rdt 2.1功能相同
      - 只是用肯定确认ACK：为ACK编号0和1
      - 发送方收到重复编号的ACK就进行重传
    - ★ 有限状态机
      - 发送方（四个状态）
      - 接收方（两个状态）
        - 收到的数据分组出错之后，就发送上一个数据分组的确认信息
    - 现实
      - 如果确认信息都是怎么办?
      - 如果数据分组丢失怎么办?
  - rdt 3.0
    - 处理信道中的bit差错和分组丢失
      - 差错控制
        - 检测
        - 确认
        - 重传
      - 分组丢失
        - 重传
        - 但如何让发送方意识到分组丢失了?
          - 发送端设置Timer
    - ★ 有限状态机
      - 发送端（四个状态）
      - 接收端（两个状态）
    - 现实
      - 效率太低
      - 大部分时间消耗在了等待中
- 实现可靠传输的基础构建
  - 检验和
  - 定时器
  - 分组序号：序号空间
    - 停止等待：1bit
    - 流水线：>1bit
  - 确认序号
    - 累计确认（GBN）
    - 重复确认（GBN和SR）
    - 逐一确认（SR）
  - 流水线
  - 滑动窗口
    - 窗口尺寸
    - 窗口变量
      - base
      - nextsqnum
- 重传协议：选择重传
  - 发送方
    - 允许发送方发送多个分组，而不需要等待
    - 发送方有选择的重传：哪个分组的ack没收到就重传哪个分组
    - ★ 发送方设置滑动窗口
  - 接收方
    - 前提
      - 对正确接受的分组进行逐个确认
      - 若分组失序，则先缓存正确分组，等待所有数据到达后再向上有序的提交数据（暂缓提交）
      - ★ 接收方设置缓存
    - 接收方怎么确认分组?
      - 分组丢失-数据失序
        - 接收方
          - 缓存失序分组
          - 逐个确认正确收到的分组
        - 发送方
          - 未收到确认信息的数据报，超时后自动重传
      - 确认丢失-超时事件
        - 发送方重传
      - 分组出错-重复确认
        - 接收方
          - 重复确认
        - 发送方
          - 收到重复确认后，传输紧临重复确认序号后的第一个数据报
- 重传协议：回退N步
  - 发送方
    - 允许发送方发送多个分组，而不需要等待确认
    - 为确认的分组数<N
      - 发送方的容量缓存有限，所有未确认的分组都必须放在缓存中
      - 不能让发送方肆无忌惮的发送数据
      - 流控制的需要
      - 拥塞控制的需要
    - 设置Timer
    - ★ 发送方设置滑动窗口
  - 接收方
    - 前提
      - 接收方没有缓存
      - 只接受有序的分组（向上提交）
      - 会丢弃失序的分组
    - 接收方怎么确认分组?
      - 分组丢失-数据失序
        - 接收方
          - 丢弃分组
        - 发送方
          - 等待Timer，进行重传
      - 确认丢失-累计确认
        - 接收方
          - 收到了数据但确认丢失
        - 发送方
          - 如果收到了ACK3，那么小于等于序号3的分组都收到了
          - 因为发送方知道接收方只接受有序的数组
      - 分组出错-重复确认
        - 接收方
          - 分组出错后，发送与最近接受到的正确的数据分组序号相同的确认信息
          - 某分组出错后分组失序，这个分组以及它后面的所有分组都会被丢弃
        - 发送方
          - 收到重复确认后，重传后面的所有数据分组
  - ★ 有限状态机
    - 发送方（一个状态）
    - 接收方（一个状态）
  - 问题
    - 接收方一下子丢失好多分组
    - 发送方可能要一下子重传好多分组
- 流水线可靠的传输协议
  - 优点：成倍地提高网络的通信效率
  - 问题：流水线中如何实现可靠控制
    - 回退N步（GBN）
    - 选择重传（SR）

TCP

- TCP协议概述
  - 因特网最复杂的协议
  - 端到端的协议（不会广播）
  - 可靠传输
    - 有序字节流传输
    - 流水线协议
  - TCP是全双工协议
    - 双工：数据流是双向的
    - 在发送是不能接收，在接收时不能发送是双工；可同时发送和接收是全双工
    - MSS：最大报文段长度（<=1480bit）
  - TCP是面向连接的协议
    - 通过握手进行同步
  - TCP可实现流控制
    - 发送方的发送速度不能超过接收方的接收速度
- TCP可靠传输
  - TCP的序号和确认号
    - 序号
      - 序号空间：0-2的32次方
      - 序号规则
        - 面向字节流编序号
          - 报文段的编号是该报文段数据中第一个字节的编号
          - Sq#（i+1）=Sq#（i）+Lsegment（i）
        - 不是对报文段编号
    - ★ 不是对报文段编号
      - ACK#（i）=Sq#（i）+Lsegment（i）
      - ACK：期望的下一个数据报
      - 理解方法
        - Seq：我是谁
        - ACK：我要请求/响应谁
        - Len：我有多长
  - TCP的确认和重传机制
    - 流水线协议，GBN和SR的混合体，接收端也有定时器！
    - TCP的确认
      - 延迟确认
        - 前提：期望序号的报文段按序到达却之前的报文段都已经被确认
        - 确认动作：延迟500ms确认新收到的分组，是累计确认机制的前提
      - 累积确认
        - 前提：期望序号的报文段按序到达，且另一按序报文正等待确认
        - 确认动作：立即发送单个累计确认，确认两个按序报文段（只能两个，不能再多了）
      - 冗余确认/重复确认
        - 前提：比期望序号大的报文段提前到达或按序到达的报文段出错
        - 确认动作：立即发送重复确认，重复确认最后一个按序到达的分组
      - 立即确认
        - 前提：失序后期望序号的报文段到达
        - 确认动作：立即发送确认信息（停止发送方这个报文段的Timer）
    - TCP的重传机制
      - 快速重传
        - 前提：发送方收到三个冗余确认
        - 重传动作：超时之前重发报文段
        - 说明网络中度拥塞
      - 加倍超时
        - 前提：发送方产生了一个超时重传
        - 重传动作：发送方加倍超时时间隔
        - 网络中发生了重度拥塞，加倍Timer，让发送方适当放缓节奏
- 往返时间估计
  - TCP协议的发送方如何设置定时器的时间间隔?
    - 时间间隔要大于（PTT）
    - 但是RTT总在波动
  - 怎样估计RTT?
    - EstimatedRTT=(1-m)*EstimatedRTT+m*SampleRTT
    - SampleRTT上一个报文段实际的RTT，m=0.125
  - 怎样估计RTT的偏差?
    - DevRTT=(1-n)*DevRTT+n*|SampleRTT-EstimateRTT|
    - n=0.25
  - 怎么设置时间间隔?
    - TimeoutInterval=EstimatedRTT +4*DevRTT
- TCP流量控制
  - 原因
    - 接收方的缓存有限
    - 接收方提交数据的速度有限
  - 目标
    - 速度匹配：发送速度不能大于接受速度
  - 机制
    - 接收窗口：缓存方缓存中的剩余空间
    - 接收方在向发送方确认时，随时告诉对方自己接收窗口的大小
- TCP连接管理
  - TCP是"面向连接"的传输协议
    - 运输前，要先建立逻辑连接
    - 客户端发起连接
    - 服务器端等待连接
    - 初始化TCP的参数
  - 建立连接
    - 三次握手（三个报文段）
  - 关闭连接
    - **四个报文段**
    - 客户端
      - 主动发送关闭连接请求
        - 进入终止等待1（FIN_WAIT_1）状态
      - 接收到服务器的确认信息
        - 进入终止等待2（FIN_WAIT_2）状态
      - 接收到服务器的释放连接通知，并且回复确认信息
        - 进入时间等待（TIMED_WAIT）状态
      - 经过两倍报文段生命周期后
        - 进入CLOSED状态
    - 服务器端
      - 接收到客户端的关闭连接请求，并且回复确认信息
        - 进入关闭等待（CLOSE_WAIT）状态
      - 传输还未传输完的数据（此时客户端虽然不能再发送数据，但可以接受数据）
      - 无数据需发送后，通知TCP释放连接
        - 进入最后确认（LAST_ACK）状态
      - 接收到客户端的确认信息
        - 进入CLOSES状态
- ★ TCP数据封装



  - 源端口号和目的端口号
    - 字段长度
      - 各占2字节
    - 主要作用
      - 端口是运输层和应用层的服务接口
      - 实现对运输层的复用和分解
  - 序号
    - 字段长度
      - 占4个字节
    - 主要作用
      - TCP连接中传送的数据流中每一个字节都会编上一个序号
      - 序号字段的值指的是本报文段所发送的数据的第一个字节的序号
  - 确认号
    - 字段长度
      - 占4个字节
    - 主要作用
      - 期望收到的报文段的序号
  - 数据偏移
    - 字段长度
      - 占4个bit
    - 主要作用
      - 数据偏移的单位是32bit=4byte=word
      - 指出TCP报文段数据起始处距离TCP数据载荷起始处有多远，即TCP首部的长度
      - 由于TCP首部有20字节的固定首部，所以数据偏移的取值范围为5—15
  - 保留字段
    - 字段长度
      - 占6bit
    - 主要作用
      - 保留给今后使用，现在全部置为0
  - URG（紧急比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当URG为1时，则告诉系统此报文段中有紧急数据，应尽快传送
      - 这里的尽快是对于不同的TCP连接而言，同一个TCP连接还是要保持数据报的不失序
  - ACK（确认比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当ACK为1时，确认号字段才有效，当ACK为0时，确认号无效
  - PSH（推送比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当PSH为1时，说明不需要接收方缓存该报文段，应尽快地交付给应用进程
      - 当PSH为1时，说明客户端使用POST方法向服务器端发送文档
  - RST（复位比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当RST为1时，表示TCP连接中出现严重差错，必须释放连接，然后再重新建立运输连接
  - SYN（同步比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当SYN为1时，表示这是一个连接请求或确认连接的报文段
  - FIN（终止比特位）
    - 字段长度
      - 占1bit
    - 主要作用
      - 当FIN为1时，表明此报文段的发送端已将数据发送完毕，并要求释放运输连接
  - 接收窗口
    - 字段长度
      - 占2个字节
    - 主要作用
      - 该窗口字段用于流控制
      - TCP连接的一端根据自己的缓存大小确定自己的接收窗口大小
      - 然后，另一端通过这个接收窗口大小设置自己的发送窗口大小
  - 检验和
    - 字段长度
      - 占2个字节
    - 主要作用
      - 检验和字段的检验范围包括首部和数据两部分
      - 在计算检验和时，要在TCP报文段的前面加上12字节的伪首部
  - 紧急指针
    - 字段长度
      - 占2个字节
    - 主要作用
      - 紧急指针指出在本报文段中的紧急数据的最后一个字节的序号
  - 选项字段
    - 字段长度
      - 可变
    - 主要作用
      - 最大报文段长度MSS，告诉对方我所能接受的最大报文段长度
  - 填充字段
    - 字段长度
      - 可变
    - 主要作用
      - 做填充，使整个首部长度是4字节的整数倍